
REIST OS – Safety Contracts und manipulationsresilienter Kontrollfluss

Technische Entwurfsnotiz / Archivfassung

Dokument-ID:	REIST-OS-TDN-EXEC-001
Version:	0.1
Datum:	16. August 2026
Status:	Konzept / Technische Entwurfsnotiz
Projekt:	REIST OS
Dokumentklasse:	Architektur- und Sicherheitskonzept
Geltungsbereich:	Kontrollfluss, Code-Integrität, Speicher- und Ausführungsautorität

Archivhinweis. Dieses Dokument hält einen konzeptionellen Entwurfsstand fest. Es beschreibt Zielbilder und mögliche Mechanismen. Die beschriebenen Verfahren sind nicht automatisch als vollständig implementiert, formal verifiziert oder zertifiziert zu verstehen. Jede spätere Umsetzung ist durch konkrete Anforderungen, Tests, Fault-Injection und nachvollziehbare Abnahmekriterien zu verifizieren.

Dokumentkontrolle

Version	Datum	Status	Änderung
0.1	16.08.2026	Konzept	Initiale Archivfassung; Safety Contracts, Buffer-Overflow-Schutz, W ^X , Code-Integrität, CFI, Shadow Stack und Abgrenzung zu Adressverschleierung zusammengeführt.

Kurzfassung

REIST OS soll Fehler nicht erst nach einem Absturz behandeln, sondern bereits die Voraussetzungen eines gültigen Kontrollflusses explizit modellieren. Eine Speicheradresse soll nicht automatisch eine Ausführungsberechtigung darstellen. Das System soll unterscheiden, ob Code an einer Adresse liegt, ob diese Adresse ein gültiger Entry Point ist, ob der aufrufende Kontext dazu berechtigt ist, ob der Code unverändert ist und ob der aktuelle Systemzustand den Kontrolltransfer zulässt.

Das Ziel ist ein mehrschichtiges Schutzmodell gegen zufällige Speicherfehler, beschädigte Function Pointer, manipulierte Rücksprungadressen, Buffer Overflows und missbräuchliche Kontrollflussänderungen. Adressrandomisierung oder Pointer-Verschleierung können ergänzende Hürden sein, ersetzen aber keine semantische Autorisierungsprüfung.

Inhaltsverzeichnis

Dokumentkontrolle	1
Kurzfassung	1
1 Zielsetzung und Grundannahme	3
2 Problemstellung	3
3 Gültige Entry Points und Call Contracts	4
3.1 Entry-Point-Validierung	4
3.2 Call Contracts	4
4 Interne, externe und indirekte Kontrolltransfers	4
4.1 Interner Call	4
4.2 Externer oder indirekter Call	5
5 Return Contracts und Shadow Stack	5
6 Control-Flow Integrity	6
7 Buffer Overflow, NX und W$\oplus$X	6
7.1 Trennung von Schreib- und Ausführungsrecht	6
7.2 W $\oplus$ X als Grundinvariante	6
8 Code-Integrität und autorisierte Ausführung	7

9 Warum Adressrandomisierung und Speicherobfuskation allein nicht genügen	7
10 Schutz vor missbrauchtem gültigem Code	8
11 Safety Contracts für Systemübergänge	8
12 Compiler- und Linker-Metadaten	9
12.1 Schutzklassen	9
13 Verbindung mit Compiler-Redundanz	10
14 Integration in das REIST-Resilienzmodell	10
15 Abgrenzung und Nicht-Ziele	11
16 Langfristiges Zielbild	11
Begriffsübersicht	12

1 Zielsetzung und Grundannahme

Dieses Dokument beschreibt einen möglichen Erweiterungsbereich von REIST OS, bei dem nicht nur Hardwarefehler, Speicherfehler oder ausfallende Dienste behandelt werden, sondern auch der **Kontrollfluss von Programmen selbst stärker abgesichert** wird.

Die zentrale Annahme lautet:

Die bloße Kenntnis einer Speicheradresse darf nicht automatisch die Berechtigung bedeuten, den dort liegenden Code auszuführen.

Ein Funktionsaufruf innerhalb eines korrekt erzeugten Programms ist semantisch etwas anderes als ein externer, indirekter oder durch Korruption entstandener Sprung auf dieselbe Adresse. Diese Fälle sollen nicht gleich behandelt werden.

Mathematisch formuliert:

$$\text{address}(x) \neq \text{authority}(x)$$

und insbesondere:

$$\text{executable}(x) \neq \text{authorized}(x)$$

2 Problemstellung

Klassische native Programme arbeiten häufig mit einem sehr direkten Maschinenmodell:

```
Adresse bekannt
-> Adresse liegt in ausführbarem Bereich
-> CALL / JMP / RET kann dorthin übertragen
```

Damit entsteht eine wichtige Fehlerklasse: Der Zielcode kann vollkommen korrekt sein, während der Weg dorthin semantisch falsch ist.

Mögliche Ursachen sind unter anderem:

- beschädigte Function Pointer,
- veränderte Rücksprungadressen,
- indirekte Calls auf falsche Ziele,
- Sprünge mitten in Funktionen,
- gültiger Code aus einem unzulässigen Caller-Kontext,
- gültige Funktionen mit ungültigen Parametern,
- Buffer Overflows, die Daten oder Kontrollinformationen überschreiben,
- zufällige Speicherfehler oder fehlerhafte DMA-Zugriffe.

Das Schutzmodell soll daher nicht nur fragen, *ob* eine Adresse ausführbar ist, sondern *warum* und *aus welchem Kontext* ein Kontrolltransfer dorthin zulässig sein soll.

3 Gültige Entry Points und Call Contracts

3.1 Entry-Point-Validierung

Nicht jede Byteadresse innerhalb einer ausführbaren Seite ist ein zulässiges Einsprungziel. Compiler und Linker können definierte Entry Points markieren.

```
Function A
0x1000  <- gueltiger Entry Point
0x1001
0x1002
0x1003
...
```

Ein indirekter Call auf 0x1000 kann zulässig sein; ein Call mitten in die Funktion, beispielsweise auf 0x1007, soll ohne explizite Freigabe verworfen werden.

3.2 Call Contracts

Zusätzlich kann ein Aufruf einen expliziten Vertrag besitzen:

```
Caller:
  StorageManager

Target:
  StorageWrite()

Allowed callers:
  StorageManager
  FileSystemService

Required authority:
  STORAGE_WRITE

Argument contract:
  buffer != NULL
  length <= 64 KiB
```

Ein dynamischer Kontrolltransfer wird damit logisch zu:

```
validate caller
validate target
validate authority
validate arguments
validate state
transfer control
```

4 Interne, externe und indirekte Kontrolltransfers

Ein direkt vom Compiler erzeugter interner Call besitzt mehr statisch bekannte Eigenschaften als ein dynamisches Ziel.

4.1 Interner Call

```

Compiler-known call site
-> known target
-> known ABI/signature
-> statically validatable edge

```

4.2 Externer oder indirekter Call

```

IPC / Plugin / Callback / Function Pointer
-> dynamic target
-> capability required
-> entry point validation
-> contract validation
-> control-flow validation

```

Für indirekte Aufrufe kann statt eines nackten Function Pointers eine **Function Capability** verwendet werden:

```

Function Capability
|- Object ID
|- Generation
|- Function Type
|- Allowed Caller Class
|- Required Privilege
|- Valid Entry Point
'- Integrity / Revocation State

```

Damit wird die Kenntnis einer Adresse von der Berechtigung zu ihrer Nutzung getrennt.

5 Return Contracts und Shadow Stack

Ein klassisches RET darf langfristig nicht lediglich bedeuten, eine Adresse vom normalen Stack zu laden und dorthin zu springen.

Ein geschützter Rücksprung kann beispielsweise prüfen:

```

current function
-> expected caller frame
-> protected return address
-> return target valid?
-> return

```

Eine mögliche Grundlage ist ein Shadow Stack:

```

normal stack:
  arguments
  local data
  return address

protected shadow stack:
  validated return address

```

Vor dem Rücksprung gilt:

$$R_{\text{stack}} = R_{\text{shadow}}$$

Nur bei Übereinstimmung wird der Return akzeptiert.

6 Control-Flow Integrity

Der Compiler kann einen zulässigen Kontrollflussgraphen erzeugen. Beispiel:

```
A -> B
A -> C
B -> D
C -> D
```

Ein Übergang wie $B \rightarrow X$, der im vorgesehenen Programmfluss nicht existiert, kann zur Laufzeit verworfen werden. Dieses Konzept ergänzt die Entry-Point-Prüfung: Ein Ziel kann ein gültiger Funktionsanfang sein und trotzdem für den aktuellen Caller unzulässig bleiben.

7 Buffer Overflow, NX und $W \oplus X$

7.1 Trennung von Schreib- und Ausführungsrecht

Ein Buffer Overflow darf nicht dazu führen, dass überschriebenes Material automatisch als Code ausgeführt werden kann. Deshalb müssen zwei Fragen getrennt werden:

1. Darf dieser Speicherbereich beschrieben werden?
2. Darf sein Inhalt ausgeführt werden?

Ein möglicher erster Schutzpfad:

```
Buffer overflow
-> bounds / guard / canary violation
-> fault detected
-> affected domain isolated
```

Falls die Speicherverletzung zunächst unentdeckt bleibt, muss eine weitere Barriere greifen:

```
data region modified
-> control transfer targets that region
-> NX / execute protection
-> execution denied
```

7.2 $W \oplus X$ als Grundinvariante

Eine Seite soll entweder schreibbar oder ausführbar sein, aber nicht beides gleichzeitig:

$$(W = 1, X = 0) \quad \vee \quad (W = 0, X = 1)$$

Nicht zulässig ist:

$$W = 1 \wedge X = 1$$

Damit wird klassischer Code-Injection entgegengewirkt. $W \oplus X$ allein verhindert jedoch keinen Missbrauch bereits vorhandenen Codes und ist daher nur eine Ebene des Gesamtmodells.

8 Code-Integrität und autorisierte Ausführung

Auch eine executable Page soll nicht allein deshalb vertrauenswürdig sein, weil sie das Execute-Bit besitzt. REIST OS kann ausführbare Bereiche beim Laden registrieren.

```
Code Page
|- Module ID
|- Generation
|- Expected integrity hash/tag
|- Valid entry points
|- Allowed call-target classes
'- Protection state
```

Für kritische Übergänge kann beispielsweise geprüft werden:

$$H(\text{Code Page}) = H_{\text{expected}}$$

Bei einer Abweichung:

```
hash mismatch
-> execution revoked
-> affected page/module fenced
-> domain isolated
-> diagnostic state captured
-> restart from trusted image
```

Die zentrale Invariante lautet:

$$\boxed{\text{Executable} \neq \text{Authorized Execution}}$$

9 Warum Adressrandomisierung und Speicherobfuskation allein nicht genügen

Moderne Systeme verwenden verschiedene Verfahren, um Speicheradressen schwerer vorhersehbar oder unmittelbar nutzbar zu machen. Dazu gehören beispielsweise ASLR, Pointer-Encoding oder Pointer-Mangling und abstrahierte Handle-Systeme.

Diese Maßnahmen sind nützlich, aber sie verändern nicht das grundlegende Autorisierungsproblem. Eine verschleierte oder zufällig platzierte Adresse ist lediglich schwerer zu erraten. Wird sie dennoch ermittelt, geleakt, rekonstruiert oder durch einen Fehler gezielt verändert, muss das System weiterhin entscheiden, ob der resultierende Kontrolltransfer semantisch zulässig ist.

Daher gilt:

Unknown Address $\neq$ Protected Control Flow

und ebenso:

Obfuscated Address $\neq$ Authorized Call

Adressverschleierung ist somit eine zusätzliche Hürde, aber keine Grundlage der Vertrauensentscheidung.

Ein REIST-konformer Kontrolltransfer soll stattdessen mindestens folgende Fragen beantworten:

- Ist das Ziel ein gültiger Entry Point?
- Ist der Caller für dieses Ziel autorisiert?
- Ist die konkrete Control-Flow-Kante erlaubt?
- Ist der Code unverändert und vertrauenswürdig?
- Ist die verwendete Capability aktuell und nicht widerrufen?
- Ist der aktuelle Systemzustand für diesen Übergang gültig?

10 Schutz vor missbrauchtem gültigem Code

NX und $W \oplus X$ adressieren primär eingeschleusten Code. Sie reichen nicht aus, wenn vorhandener legitimer Code missbraucht wird, etwa durch:

- Return-Oriented Programming,
- Jump-Oriented Programming,
- manipulierte Rücksprungadressen,
- beschädigte Function Pointer,
- unzulässige indirekte Calls.

Deshalb sollen mehrere Schutzebenen zusammenwirken:

$NX/W \oplus X + \text{Shadow Stack} + \text{CFI} + \text{Call Contracts} + \text{Entry-Point Validation}$
--

Keine einzelne Ebene wird als ausreichend betrachtet.

11 Safety Contracts für Systemübergänge

Das Konzept kann über normale Funktionsaufrufe hinaus erweitert werden. Eigene Contracts sind denkbar für:

- CALL,
- RETURN,

- INDIRECT CALL,
- SYSCALL,
- IPC,
- INTERRUPT,
- EXCEPTION,
- DEVICE CALLBACK,
- SERVICE RESTART.

Beispiel für einen System Call:

```
Userspace
-> valid syscall entry?
-> valid process capability?
-> arguments valid?
-> resource authority valid?
-> kernel operation
```

Ein Userspace-Prozess darf niemals dadurch Kernelautorität erlangen, dass er lediglich eine Kerneladresse kennt.

12 Compiler- und Linker-Metadaten

Compiler und Linker können zusätzliche Safety-Metadaten erzeugen:

```
Function ID
Function Type
Allowed Callers
Privilege Class
Entry Point
Expected ABI
Stack Requirements
Argument Constraints
Return Contract
Safety Level
Code Integrity Tag
```

Diese Metadaten können im Executable abgelegt, beim Laden verifiziert und teilweise in geschützte Kernelstrukturen übernommen werden.

12.1 Schutzklassen

Nicht jeder Codepfad benötigt denselben Aufwand. Ein mögliches Modell:

Level	Beispielhafte Schutzklasse
0	Normaler Code ohne zusätzliche REIST-Verträge
1	Gültige Entry Points
2	Control-Flow-Prüfung
3	Capability + Call Contract
4	Shadow Stack + redundanter Kontrollfluss
5	Vollständig kritischer Safety-Pfad mit maximaler Validierung

Dadurch kann Schutzaufwand gezielt gegen Performance, Speicherbedarf und Komplexität abgewogen werden.

13 Verbindung mit Compiler-Redundanz

Safety Contracts lassen sich mit compilerseitiger Fehlertoleranz kombinieren. Ein kritischer Pfad könnte beispielsweise so aussehen:

```
validated input
-> redundant computation
-> compare results
-> validate control flow
-> verify code integrity
-> verify semantic invariant
-> commit
```

Das daraus entstehende Modell kombiniert mehrere unabhängige Schutzarten:

Memory Safety + $W^{\oplus}X$ + Code Integrity + CFI + Shadow Stack + Execution Contracts

Adressrandomisierung oder Obfuskation kann ergänzend hinzukommen, ist aber ausdrücklich nicht Teil der eigentlichen Autorisierungsentscheidung.

14 Integration in das REIST-Resilienzmodell

Die bestehenden REIST-Prinzipien lassen sich direkt auf Kontrollflussfehler übertragen:

```
Detect
-> Contain
-> Recover
-> Validate
-> Reintegrate
```

Beispiel eines ungültigen indirekten Calls:

```
invalid indirect call
-> detect violation
-> prevent transfer
-> isolate affected domain
-> capture fault context
-> restart service/domain
-> validate code and state
```

```
-> reintegrate
```

Ein ungültiger Kontrolltransfer muss somit nicht automatisch zu einem globalen Systemfehler führen.

15 Abgrenzung und Nicht-Ziele

Die beschriebenen Mechanismen erhöhen sowohl Fehlertoleranz als auch Manipulationsresilienz. Sie ersetzen jedoch kein vollständiges Security-Modell und garantieren keine absolute Sicherheit.

Insbesondere sind weiterhin separate Maßnahmen erforderlich für Themen wie:

- kryptographische Authentisierung und Vertraulichkeit,
- sichere Schlüsselverwaltung,
- Supply-Chain-Sicherheit,
- physische Manipulation,
- Seitenkanäle,
- Compiler- oder Toolchain-Kompromittierung,
- Common-Mode-Fehler in mehreren Schutzschichten.

Das Ziel dieses Dokuments ist enger: Kontrollfluss, Codezustand und Ausführungsautorität sollen so modelliert werden, dass die bloße Manipulation einer Adresse oder eines Speicherslots nicht automatisch in legitime Codeausführung übergeht.

16 Langfristiges Zielbild

Langfristig soll ein Programm nicht lediglich aus frei adressierbaren Codebytes bestehen. Der Kontrollfluss soll eine explizite Semantik besitzen:

Adresse + Entry Point + Capability + Contract + Code Integrity + State = gültiger Kontrolltransfer
--

Der grundlegende Gedanke lautet:

Ein Programm soll nicht nur wissen, wohin es springen kann. Das System soll auch wissen, wohin es springen darf, aus welchem Kontext dieser Sprung zulässig ist und ob der dort liegende Code noch genau der Code ist, dem es ursprünglich vertraut hat.

Adressverschleierung kann einen Fehler oder Angreifer bremsen. Safety Contracts sollen dagegen verhindern, dass die bloße Kenntnis oder Manipulation einer Adresse überhaupt ausreicht.

Begriffsübersicht

Begriff	Bedeutung im Dokument
Entry Point	Explizit zugelassener Einsprungpunkt in ausführbaren Code.
Capability	Nicht bloß eine Adresse, sondern eine überprüfbare Ausführungs- oder Ressourcenberechtigung.
Call Contract	Regeln für Caller, Ziel, Signatur, Argumente, Privilegien und Zustand.
CFI	Control-Flow Integrity; Prüfung, ob Kontrolltransfers im erlaubten Graphen liegen.
Shadow Stack	Separat geschützte Referenz für Rücksprungadressen.
$W \oplus X$	Speicher soll entweder beschreibbar oder ausführbar sein, nicht beides zugleich.
Code Integrity	Nachweis, dass ausführbarer Code dem erwarteten, freigegebenen Zustand entspricht.
Fail-closed	Bei Unsicherheit wird Ausführung oder Autorität verweigert statt stillschweigend erlaubt.